

Practical 1 - ER Diagram and Create Tables

```
-- =====
-- PRACTICAL 1 - ER Diagram and Create Tables
-- =====

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

-- Create DEPARTMENT first (no foreign keys)
CREATE TABLE DEPARTMENT (
DeptNo VARCHAR(5) PRIMARY KEY,
DeptName VARCHAR(30) NOT NULL,
HOD VARCHAR(30)
);

-- Create STUDENT (depends on DEPARTMENT)
CREATE TABLE STUDENT (
RollNo INT PRIMARY KEY,
Name VARCHAR(30) NOT NULL,
Email VARCHAR(50) UNIQUE,
DeptNo VARCHAR(5),
FOREIGN KEY (DeptNo) REFERENCES DEPARTMENT(DeptNo)
);

-- Create COURSE (depends on DEPARTMENT)
CREATE TABLE COURSE (
CourseID VARCHAR(5) PRIMARY KEY,
CourseName VARCHAR(30) NOT NULL,
Credits INT,
DeptNo VARCHAR(5),
FOREIGN KEY (DeptNo) REFERENCES DEPARTMENT(DeptNo)
);

-- Create ENROLLMENT (depends on STUDENT and COURSE)
CREATE TABLE ENROLLMENT (
RollNo INT,
CourseID VARCHAR(5),
EnrollDate DATE,
Grade VARCHAR(2),
PRIMARY KEY (RollNo, CourseID),
FOREIGN KEY (RollNo) REFERENCES STUDENT(RollNo),
FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
);

-- Verify tables created
SHOW TABLES;
```

Practical 2 - DDL Commands

```
-- =====
-- PRACTICAL 2 - DDL Commands
-- =====

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

-- Create DEPARTMENT first (needed for foreign key in STUDENT)
CREATE TABLE DEPARTMENT (
DeptNo VARCHAR(5) PRIMARY KEY,
DeptName VARCHAR(30) NOT NULL,
HOD VARCHAR(30)
);

-- Create STUDENT with all constraints
CREATE TABLE STUDENT (
RollNo INT PRIMARY KEY,
Name VARCHAR(30) NOT NULL,
Email VARCHAR(50) UNIQUE,
Age INT CHECK (Age >= 17),
DeptNo VARCHAR(5),
FOREIGN KEY (DeptNo) REFERENCES DEPARTMENT(DeptNo)
);

-- Insert into DEPARTMENT
INSERT INTO DEPARTMENT VALUES ('D01', 'Computer Engg', 'Dr. Sharma');
INSERT INTO DEPARTMENT VALUES ('D02', 'Electronics', 'Dr. Patil');

-- Insert into STUDENT
INSERT INTO STUDENT VALUES (101, 'Shreyas', 'shreyas@mail.com', 19, 'D01');
INSERT INTO STUDENT VALUES (102, 'Yash', 'yash@mail.com', 20, 'D01');
INSERT INTO STUDENT VALUES (103, 'Tejas', 'tejas@mail.com', 18, 'D02');

-- View data
SELECT * FROM STUDENT;

-- ALTER: Add new column
ALTER TABLE STUDENT ADD COLUMN Phone VARCHAR(15);

-- ALTER: Modify column size
ALTER TABLE STUDENT MODIFY COLUMN Name VARCHAR(50);

-- View updated structure
DESCRIBE STUDENT;

-- Rename table
RENAME TABLE STUDENT TO STUDENTS;

-- Truncate: delete all rows, keep structure
TRUNCATE TABLE STUDENTS;

-- Drop: delete table completely
```

DROP TABLE STUDENTS;

Practical 3 - DML, DCL and TCL Commands

```
-- =====
-- PRACTICAL 3 - DML, DCL and TCL Commands
-- =====

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

-- Setup tables first
CREATE TABLE DEPARTMENT (
DeptNo VARCHAR(5) PRIMARY KEY,
DeptName VARCHAR(30) NOT NULL,
HOD VARCHAR(30)
);

CREATE TABLE STUDENT (
RollNo INT PRIMARY KEY,
Name VARCHAR(30) NOT NULL,
Email VARCHAR(50) UNIQUE,
Age INT CHECK (Age >= 17),
DeptNo VARCHAR(5),
Phone VARCHAR(15),
FOREIGN KEY (DeptNo) REFERENCES DEPARTMENT(DeptNo)
);

INSERT INTO DEPARTMENT VALUES ('D01', 'Computer Engg', 'Dr. Sharma');
INSERT INTO DEPARTMENT VALUES ('D02', 'Electronics', 'Dr. Patil');

INSERT INTO STUDENT VALUES (101, 'Shreyas', 'shreyas@mail.com', 19, 'D01', NULL);
INSERT INTO STUDENT VALUES (102, 'Yash', 'yash@mail.com', 20, 'D01', NULL);
INSERT INTO STUDENT VALUES (103, 'Tejas', 'tejas@mail.com', 18, 'D02', NULL);

-- =====
-- a) DML Commands
-- =====

-- INSERT
INSERT INTO STUDENT VALUES (104, 'Radha', 'radha@mail.com', 19, 'D01', NULL);
INSERT INTO STUDENT VALUES (105, 'Krishna', 'krishna@mail.com', 21, 'D02', NULL);

SELECT * FROM STUDENT;

-- UPDATE
UPDATE STUDENT SET Age = 21 WHERE RollNo = 101;

-- DELETE
DELETE FROM STUDENT WHERE RollNo = 103;

-- SELECT with LIKE
SELECT * FROM STUDENT WHERE Name LIKE 'S%';

-- Arithmetic in SELECT
SELECT Name, Age, (Age + 1) AS NextAge FROM STUDENT;
```

```

-- String functions
SELECT UPPER(Name) AS UpperName, LENGTH(Name) AS NameLen FROM STUDENT;

-- Date functions
SELECT NOW(), CURDATE();

-- =====
-- b) DCL Commands
-- =====

CREATE USER 'testuser'@'localhost' IDENTIFIED BY 'pass123';
GRANT SELECT, INSERT ON college_db.STUDENT TO 'testuser'@'localhost';
SHOW GRANTS FOR 'testuser'@'localhost';
REVOKE INSERT ON college_db.STUDENT FROM 'testuser'@'localhost';

-- =====
-- c) TCL Commands
-- =====

START TRANSACTION;

INSERT INTO STUDENT VALUES (106, 'Shreyas', 'shreyas2@mail.com', 20, 'D01', NULL);
UPDATE STUDENT SET Age = 22 WHERE RollNo = 102;
SAVEPOINT sp1;
DELETE FROM STUDENT WHERE RollNo = 104;
ROLLBACK TO SAVEPOINT sp1;
SELECT * FROM STUDENT;
COMMIT;

```

Practical 4 - Aggregate Functions and Grouping

```
-- =====
-- PRACTICAL 4 - Aggregate Functions and Grouping
-- =====

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

-- Create and populate STUDENT_MARKS table
CREATE TABLE STUDENT_MARKS (
RollNo INT,
Name VARCHAR(30),
DeptNo VARCHAR(5),
Marks INT,
Subject VARCHAR(20)
);

INSERT INTO STUDENT_MARKS VALUES (101, 'Shreyas', 'D01', 72, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (102, 'Yash', 'D01', 85, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (103, 'Tejas', 'D02', 60, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (104, 'Radha', 'D01', 90, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (105, 'Krishna', 'D02', 55, 'DBMS');

-- View all data
SELECT * FROM STUDENT_MARKS;

-- COUNT: total number of students
SELECT COUNT(*) AS TotalStudents FROM STUDENT_MARKS;

-- SUM: total marks of all students
SELECT SUM(Marks) AS TotalMarks FROM STUDENT_MARKS;

-- AVG: average marks
SELECT AVG(Marks) AS AverageMarks FROM STUDENT_MARKS;

-- MAX and MIN
SELECT MAX(Marks) AS Highest, MIN(Marks) AS Lowest FROM STUDENT_MARKS;

-- GROUP BY: count and total per department
SELECT DeptNo, COUNT(*) AS StudentCount, SUM(Marks) AS Total
FROM STUDENT_MARKS
GROUP BY DeptNo;

-- HAVING: only show departments where average marks > 65
SELECT DeptNo, AVG(Marks) AS AvgMarks
FROM STUDENT_MARKS
GROUP BY DeptNo
HAVING AVG(Marks) > 65;
```

Practical 5 - JOIN Operations and Views

```
-- =====
-- PRACTICAL 5 - JOIN Operations and Views
-- =====

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

-- Create tables
CREATE TABLE DEPARTMENT (
DeptNo VARCHAR(5) PRIMARY KEY,
DeptName VARCHAR(30) NOT NULL,
HOD VARCHAR(30)
);

CREATE TABLE STUDENT (
RollNo INT PRIMARY KEY,
Name VARCHAR(30) NOT NULL,
Email VARCHAR(50) UNIQUE,
Age INT,
DeptNo VARCHAR(5),
Phone VARCHAR(15),
FOREIGN KEY (DeptNo) REFERENCES DEPARTMENT(DeptNo)
);

-- Insert data
INSERT INTO DEPARTMENT VALUES ('D01', 'Computer Engg', 'Dr. Sharma');
INSERT INTO DEPARTMENT VALUES ('D02', 'Electronics', 'Dr. Patil');
INSERT INTO DEPARTMENT VALUES ('D03', 'Mechanical', 'Dr. More');

INSERT INTO STUDENT VALUES (101, 'Shreyas', 'shreyas@mail.com', 19, 'D01', NULL);
INSERT INTO STUDENT VALUES (102, 'Yash', 'yash@mail.com', 20, 'D01', NULL);
INSERT INTO STUDENT VALUES (103, 'Tejas', 'tejas@mail.com', 18, 'D02', NULL);
INSERT INTO STUDENT VALUES (104, 'Radha', 'radha@mail.com', 19, 'D01', NULL);
INSERT INTO STUDENT VALUES (105, 'Krishna', 'krishna@mail.com', 21, 'D02', NULL);
-- This student has no department (to show LEFT JOIN)
INSERT INTO STUDENT VALUES (106, 'Yash', 'yash2@mail.com', 22, NULL, NULL);

-- INNER JOIN: only students who have a matching department
SELECT S.RollNo, S.Name, D.DeptName
FROM STUDENT S
INNER JOIN DEPARTMENT D ON S.DeptNo = D.DeptNo;

-- LEFT JOIN: all students, even those with no department
SELECT S.RollNo, S.Name, D.DeptName
FROM STUDENT S
LEFT JOIN DEPARTMENT D ON S.DeptNo = D.DeptNo;

-- RIGHT JOIN: all departments, even those with no students
SELECT S.Name, D.DeptNo, D.DeptName
```

```
FROM STUDENT S
RIGHT JOIN DEPARTMENT D ON S.DeptNo = D.DeptNo;

-- Create VIEW
CREATE VIEW CS_STUDENTS AS
SELECT RollNo, Name
FROM STUDENT
WHERE DeptNo = 'D01';

-- Use the view
SELECT * FROM CS_STUDENTS;

-- Drop the view
DROP VIEW CS_STUDENTS;
```


Practical 6 - Subqueries

```
-- =====
-- PRACTICAL 6 - Subqueries
-- =====

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

-- Create and populate table
CREATE TABLE STUDENT_MARKS (
RollNo INT,
Name VARCHAR(30),
DeptNo VARCHAR(5),
Marks INT,
Subject VARCHAR(20)
);

INSERT INTO STUDENT_MARKS VALUES (101, 'Shreyas', 'D01', 72, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (102, 'Yash', 'D01', 85, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (103, 'Tejas', 'D02', 60, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (104, 'Radha', 'D01', 90, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (105, 'Krishna', 'D02', 55, 'DBMS');

-- 1. Students scoring above average marks
SELECT Name, Marks
FROM STUDENT_MARKS
WHERE Marks > (SELECT AVG(Marks) FROM STUDENT_MARKS);

-- 2. Student with the highest marks
SELECT Name, Marks
FROM STUDENT_MARKS
WHERE Marks = (SELECT MAX(Marks) FROM STUDENT_MARKS);

-- 3. Students from the same department as Shreyas
SELECT Name
FROM STUDENT_MARKS
WHERE DeptNo = (SELECT DeptNo FROM STUDENT_MARKS WHERE Name = 'Shreyas');

-- 4. Subquery using IN
SELECT Name, Marks
FROM STUDENT_MARKS
WHERE RollNo IN (101, 102, 104);

-- 5. Subquery using NOT IN
SELECT Name, Marks
FROM STUDENT_MARKS
WHERE RollNo NOT IN (103, 105);
```

Practical 7 - Stored Procedure, Cursor and Function

```
-- =====
-- PRACTICAL 7 - Stored Procedure, Cursor and Function
-- =====

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

-- Create and populate table
CREATE TABLE STUDENT_MARKS (
RollNo INT,
Name VARCHAR(30),
DeptNo VARCHAR(5),
Marks INT,
Subject VARCHAR(20)
);

INSERT INTO STUDENT_MARKS VALUES (101, 'Shreyas', 'D01', 72, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (102, 'Yash', 'D01', 85, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (103, 'Tejas', 'D02', 60, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (104, 'Radha', 'D01', 90, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (105, 'Krishna', 'D02', 55, 'DBMS');

-- Change delimiter so MySQL does not confuse ; inside procedure
DELIMITER //

-- Stored Procedure using Cursor
CREATE PROCEDURE ShowStudentMarks()
BEGIN
DECLARE v_name VARCHAR(30);
DECLARE v_marks INT;
DECLARE done INT DEFAULT 0;

-- Declare cursor
DECLARE cur CURSOR FOR
SELECT Name, Marks FROM STUDENT_MARKS;

-- Handler: when no more rows, set done = 1
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

OPEN cur;

read_loop: LOOP
FETCH cur INTO v_name, v_marks;
IF done = 1 THEN
LEAVE read_loop;
END IF;
SELECT v_name AS Name, v_marks AS Marks;
END LOOP;

CLOSE cur;
END //
```

```
-- Function to return grade based on marks
CREATE FUNCTION GetGrade(marks INT)
RETURNS VARCHAR(2)
DETERMINISTIC
BEGIN
DECLARE grade VARCHAR(2);
IF marks >= 75 THEN
SET grade = 'A';
ELSEIF marks >= 60 THEN
SET grade = 'B';
ELSEIF marks >= 50 THEN
SET grade = 'C';
ELSE
SET grade = 'F';
END IF;
RETURN grade;
END //

-- Restore delimiter back to normal
DELIMITER ;

-- Call the procedure
CALL ShowStudentMarks();

-- Use the function in a SELECT query
SELECT Name, Marks, GetGrade(Marks) AS Grade FROM STUDENT_MARKS;
```

Practical 8 - Database Triggers

```
-- =====
-- PRACTICAL 8 - Database Triggers
-- =====

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

-- Create STUDENT_MARKS table
CREATE TABLE STUDENT_MARKS (
RollNo INT,
Name VARCHAR(30),
DeptNo VARCHAR(5),
Marks INT,
Subject VARCHAR(20)
);

INSERT INTO STUDENT_MARKS VALUES (101, 'Shreyas', 'D01', 72, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (102, 'Yash', 'D01', 85, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (103, 'Tejas', 'D02', 60, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (104, 'Radha', 'D01', 90, 'DBMS');
INSERT INTO STUDENT_MARKS VALUES (105, 'Krishna', 'D02', 55, 'DBMS');

-- Create LOG table to store changes
CREATE TABLE MARKS_LOG (
LogID INT AUTO_INCREMENT PRIMARY KEY,
RollNo INT,
OldMarks INT,
NewMarks INT,
ChangedAt DATETIME
);

DELIMITER //

-- BEFORE UPDATE trigger: runs before any UPDATE on STUDENT_MARKS
CREATE TRIGGER before_marks_update
BEFORE UPDATE ON STUDENT_MARKS
FOR EACH ROW
BEGIN
INSERT INTO MARKS_LOG (RollNo, OldMarks, NewMarks, ChangedAt)
VALUES (OLD.RollNo, OLD.Marks, NEW.Marks, NOW());
END //

-- AFTER INSERT trigger: runs after any INSERT into STUDENT_MARKS
CREATE TRIGGER after_student_insert
AFTER INSERT ON STUDENT_MARKS
FOR EACH ROW
BEGIN
INSERT INTO MARKS_LOG (RollNo, OldMarks, NewMarks, ChangedAt)
VALUES (NEW.RollNo, 0, NEW.Marks, NOW());
```

```
END //

DELIMITER ;

-- Test BEFORE UPDATE trigger
UPDATE STUDENT_MARKS SET Marks = 78 WHERE RollNo = 101;

-- Check log table
SELECT * FROM MARKS_LOG;

-- Test AFTER INSERT trigger
INSERT INTO STUDENT_MARKS VALUES (106, 'Radha', 'D01', 88, 'DBMS');

-- Check log table again
SELECT * FROM MARKS_LOG;

-- Show all triggers
SHOW TRIGGERS;
```

Practical 9 - MongoDB CRUD Operations

```
// =====
// PRACTICAL 9 - MongoDB CRUD Operations
// =====
// Open MongoDB by typing: mongosh
// Then copy and run these commands one by one

// Select database
use college_db

// Drop collection if exists (fresh start)
db.students.drop()

// INSERT one document
db.students.insertOne({
  rollno: 101,
  name: "Shreyas",
  dept: "Computer Engg",
  marks: 72
})

// INSERT multiple documents
db.students.insertMany([
  { rollno: 102, name: "Yash", dept: "Computer Engg", marks: 85 },
  { rollno: 103, name: "Tejas", dept: "Electronics", marks: 60 },
  { rollno: 104, name: "Radha", dept: "Computer Engg", marks: 90 },
  { rollno: 105, name: "Krishna", dept: "Electronics", marks: 55 }
])

// READ: find all documents
db.students.find()

// READ: find with condition (marks greater than 70)
db.students.find({ marks: { $gt: 70 } })

// READ: find one document
db.students.findOne({ rollno: 101 })

// UPDATE: update marks of rollno 101
db.students.updateOne(
  { rollno: 101 },
  { $set: { marks: 78 } }
)

// Verify update
db.students.findOne({ rollno: 101 })

// DELETE: delete rollno 103
db.students.deleteOne({ rollno: 103 })

// Verify final data
db.students.find()
```

Practical 10 - MongoDB Aggregation and Indexing

```
// =====
// PRACTICAL 10 - MongoDB Aggregation and Indexing
// =====
// Open MongoDB by typing: mongosh
// Then copy and run these commands one by one

// Select database
use college_db

// Drop and recreate collection (fresh start)
db.students.drop()

db.students.insertMany([
{ rollno: 101, name: "Shreyas", dept: "Computer Engg", marks: 72 },
{ rollno: 102, name: "Yash", dept: "Computer Engg", marks: 85 },
{ rollno: 103, name: "Tejas", dept: "Electronics", marks: 60 },
{ rollno: 104, name: "Radha", dept: "Computer Engg", marks: 90 },
{ rollno: 105, name: "Krishna", dept: "Electronics", marks: 55 }
])

// =====
// a) Aggregation
// =====

// Count total documents
db.students.aggregate([
{ $count: "TotalStudents" }
])

// Group by dept: count, avg, max, min marks
db.students.aggregate([
{
  $group: {
    _id: "$dept",
    TotalStudents: { $sum: 1 },
    AvgMarks: { $avg: "$marks" },
    MaxMarks: { $max: "$marks" },
    MinMarks: { $min: "$marks" }
  }
}
])

// Filter (marks > 60) then group by dept
db.students.aggregate([
{ $match: { marks: { $gt: 60 } } },
{
  $group: {
    _id: "$dept",
    Count: { $sum: 1 }
  }
}
```

```

}
])

// Sort by marks descending, show only name and marks
db.students.aggregate([
{ $sort: { marks: -1 } },
{ $project: { _id: 0, name: 1, marks: 1 } }
])

// =====
// b) Indexing
// =====

// Create index on rollno
db.students.createIndex({ rollno: 1 })

// Create index on name
db.students.createIndex({ name: 1 })

// Create compound index on dept and marks
db.students.createIndex({ dept: 1, marks: -1 })

// See all indexes
db.students.getIndexes()

// Drop an index
db.students.dropIndex("name_1")

// Search using indexed field
db.students.find({ rollno: 102 })

```